

Operator: `argmax`

Phase 1.5 nấc 2 → đầu vào cho Phase 2A (viết operator) và Phase 2D (golden model C).

Điểm kết thúc của inference. Sau operator này không còn gì nữa.

1. Vai trò

Nhận 10 logit, trả về **chỉ số** của số lớn nhất. Chỉ số đó chính là chữ số dự đoán: chỉ số 7 nghĩa là model đoán "7".

Không trả về giá trị, trả về **vị trí**. Đây là chỗ mạng chuyển từ "10 con số thực" sang "một câu trả lời".

2. Công thức

```
argmax(z) = chỉ số i sao cho z[i] ≥ z[j] với mọi j
```

Vì sao không cần softmax

Công thức softmax:

```
softmax(z)[i] = exp(z[i]) / Σ exp(z[j])
```

`exp` là hàm **đơn điệu tăng nghiêm ngặt**, và mẫu số là cùng một hằng số dương cho cả 10 phần tử. Nên softmax **giữ nguyên thứ tự**: `z[a] > z[b]` thì `softmax(z)[a] > softmax(z)[b]`.

Suy ra:

```
argmax( softmax(z) ) = argmax(z)
```

→ **Bỏ hẳn softmax trên chip**. Tiết kiệm 10 phép `exp()` và 1 phép chia — `exp` là phép đắt nhất trong toàn mạng nếu phải hiện thực (thường phải dùng bảng tra hoặc xấp xỉ đa thức trên FPGA). Đúng yêu cầu "thay/bỏ phép toán khó cho FPGA" của project.

Vì sao model không có sẵn lớp softmax

`SmallCNN.forward()` kết thúc bằng `return self.fc(x)` — logit thô, không softmax. Đó không phải thiếu sót.

Lúc train dùng `nn.CrossEntropyLoss`, mà hàm này = `LogSoftmax` + `NLLLoss` gộp lại, **tự áp softmax bên trong**. Nếu thêm softmax vào `forward()` nữa thì bị áp hai lần, loss sai, model học kém đi.

Kết quả tiện lợi: model viết đúng chuẩn train thì đồng thời đã sẵn sàng cho phần cứng, không phải gỡ lớp nào ra.

3. Mã giả

```
argmax(logits[10]):

best_idx = 0
best_val = logits[0]

for i = 1 .. 9:
    if logits[i] > best_val:          # ">" chứ không phải ">=" — xem mục 6
        best_val = logits[i]
        best_idx = i

return best_idx
```

9 phép so sánh. Không nhân, không chia, không cộng.

4. Áp dụng vào `SmallCNN`

Giá trị	
Đầu vào	10 logit (float32, hoặc int32 nếu chạy fixed-point)
Đầu ra	1 số nguyên trong khoảng 0–9
Params	0
Phép so sánh	9

Đặt cạnh 395.136 MAC của phần conv: argmax chiếm khoảng **0,002%** khối lượng tính toán.

5. Mức FPGA-friendly

✅ **Gần như miễn phí.** Một cây comparator: 10 đầu vào gộp đôi thành 5, rồi 3, rồi 2, rồi 1 — độ sâu 4 tầng. Không DSP, không bộ nhân, vài chục LUT.

So sánh thẳng trên **int32 accumulator** được, không cần dequantize về float trước. Vì phép dequantize là nhân với cùng một `scale` dương cho cả 10 logit, mà nhân với hằng số dương cũng giữ nguyên thứ tự — y hệt lý do bỏ được softmax. Bớt thêm 10 phép nhân nữa.

6. Chú ý khi port sang C

Bẫy	Chi tiết
** > hay >= **	Dùng > thì khi hoà chọn chỉ số nhỏ nhất ; dùng >= thì chọn lớn nhất . <code>torch.argmax</code> trả về chỉ số nhỏ nhất → phải viết > . Sai chỗ này thì golden model lệch PyTorch ở đúng những ảnh bị hoà

Bẫy	Chi tiết
Hoà không hiểm như tưởng	Với float32 hầu như không bao giờ hoà. Nhưng sau khi lượng tử hoá, giá trị bị dồn về lưới — xem <code>quantization.py</code> bảng 3 cho thấy 2.304 trọng số <code>conv3</code> chỉ còn 155 giá trị khác nhau ở int8. Logit cũng bị dồn tương tự, và khả năng hai logit trùng nhau tăng lên rõ rệt
Đừng hiện thực softmax	Xem mục 2. Nếu code C có <code>exp()</code> ở lớp cuối thì đang làm thừa
Nếu cần độ tin cậy	Lúc đó mới buộc phải có softmax, và <code>exp()</code> quay lại. Cách rẻ hơn: xuất hiệu giữa logit lớn nhất và logit lớn nhì làm thước đo "chắc chắn tới đâu" — chỉ tốn 1 phép trừ
Khởi tạo	<code>best_idx = 0, best_val = logits[0]</code> . Đừng khởi tạo <code>best_val</code> bằng 0 — logit có thể âm hết, khi đó hàm trả về sai

7. Tự kiểm tra

Làm trên giấy:

1. Chứng minh `argmax(softmax(z)) = argmax(z)` bằng tính đơn điệu. Cần điều kiện gì về mẫu số?
2. Sau khi lượng tử hoá int8, khả năng hai logit bằng nhau tăng hay giảm? Giải thích dựa vào bảng 3 của `quantization.py`.
3. Nếu đổi `>` thành `>=` trong mã giả, loại ảnh nào có thể đổi kết quả? Ước lượng trong 10.000 ảnh test thì có bao nhiêu ảnh như vậy ở float32, và ở int8?
4. Logit của một ảnh là `[-2.1, 0.4, 8.7, 8.7, -1.0, 3.2, 0.0, -5.5, 1.1, 2.9]`. `torch.argmax` trả về gì? Code C viết `>=` trả về gì?

Đáp án

Câu 1 — Chứng minh bằng tính đơn điệu

$\text{softmax}(z)_i = \exp(z_i) / S$ với $S = \sum_j \exp(z_j)$.

1. **`exp` đơn điệu tăng nghiêm ngặt**: $z_i > z_j \iff \exp(z_i) > \exp(z_j)$. Thứ tự giữ nguyên, dấu bằng cũng giữ nguyên.
2. **`S` không phụ thuộc `i`** — nó là tổng trên mọi `j`, nên cả 10 phần tử chia cho cùng một số.
3. **`S > 0`** vì $\exp(x) > 0$ với mọi `x`.

Chia mọi phần tử cho cùng một hằng số dương thì thứ tự không đổi → `argmax` trùng nhau. Đây là đẳng thức chắc chắn, không phải xấp xỉ.

Điều kiện về mẫu số — hai điều, thiếu một là sai:

Điều kiện	Nếu vi phạm
Giống nhau cho mọi lớp	Mỗi lớp chia một số khác nhau → thứ tự đảo tùy ý, chứng minh sập
Dương	Chia cho số âm thì bất đẳng thức đổi chiều → <code>argmax</code> thành <code>argmin</code>

Cả hai thoả tự động với softmax.

Lý do thực tế thứ hai để bỏ softmax: logit lớn nhất đo được của model là **60,356**. `exp(60,356) ≈ 1,6 · 1026` — vẫn lọt float32 (trần ~3,4 · 10³⁸), nhưng logit tới ~89 là trần. Bỏ softmax thì không phải lo chuyện đó trên chip.

Câu 2 — Tăng, nhưng bảng 3 không trực tiếp chứng minh điều đó

Bảng 3 ("Độ phân giải") cho thấy: float32 gần như mỗi trọng số một giá trị riêng, int8 tối đa 255 giá trị. Đó là **nguyên lý chuồng bồ câu** — nén nhiều giá trị vào ít ô thì phải có trùng.

Chỗ dễ trả lời hớ: bảng 3 đếm trùng trên _trọng số_, không phải trên _logit_.

`xem_quantization.py` chỉ lượng tử hoá weight-only — trọng số bị ép về lưới rồi *giải lượng tử về float32* để nhân, nên logit đầu ra vẫn là float32 đầy đủ độ phân giải. Chính file đó đã ghi: "*Script này chỉ lượng tử hoá TRONG SỐ, còn activation giữa các lớp vẫn là float32.*"

Hoà logit chỉ tăng thật khi lượng tử hoá **cả đường inference** — activation int8, accumulator int32 — vì lúc đó logit là **số nguyên** trong dải hẹp.

Số đo (`study/dem_hoa_logit.py`, 10.000 ảnh test):

Kiểu số	Số ảnh hoà	Khoảng cách nhỏ nhất giữa 2 logit cao nhất
float32 gốc	0	1,900e-02
int8 weight-only	0	7,344e-03 — hẹp lại 2,6 lần
Ép logit về lưới 8-bit (mô phỏng)	7	0

Weight-only chưa tạo hoà nào, nhưng cơ chế đã lộ: khoảng cách co lại 2,6 lần.

Câu 3 — Ảnh có cực đại đạt tại từ 2 lớp trở lên

Chỉ loại ảnh đó mới đổi kết quả:

- `>` không cập nhật khi bằng → giữ **chỉ số nhỏ nhất**
- `>=` cập nhật cả khi bằng → giữ **chỉ số lớn nhất**

Ảnh có một cực đại duy nhất thì hai cách cho cùng kết quả.

Số đo, không phải ước lượng:

Kiểu số	Số ảnh đổi kết quả / 10.000
float32 gốc	0
int8 weight-only	0
Ép logit về lưới 8-bit	7 (0,07%)

Câu 4 — `torch.argmax` → 2, code C viết `>=` → 3

Giá trị lớn nhất là `8.7`, đạt tại **hai** chỉ số: 2 và 3.

Cách	Trả về	Vì sao
<code>torch.argmax</code>	2	Trả về chỉ số đầu tiên đạt cực đại
C viết <code>></code> (đúng)	2	<code>8.7 > 8.7</code> sai → không cập nhật, giữ 2
C viết <code>>=</code> (sai)	3	<code>8.7 >= 8.7</code> đúng → cập nhật <code>best = 3</code>

Điểm chính của câu này: giả sử đáp án thật của ảnh là 7 — cả hai cách đều đoán sai, **accuracy giống hệt nhau**, bug `>=` hoàn toàn vô hình. Đây đúng loại lỗi "không crash, chỉ ra kết quả sai", và là lý do tiêu chí verify phải là so từng phần tử chứ không phải so accuracy cuối.

Biên an toàn rút ra từ ba câu trên

Đại lượng	Giá trị
Khoảng cách nhỏ nhất giữa 2 logit cao nhất (float32)	1,900e-02
Sai số Linear ↔ Conv2d đo được (<code>study/kiem_linear_vs_conv.py</code>)	2,174e-04
Dư địa	87 lần

Đây là lý do lệch `2,17e-04` mà 0/10.000 ảnh đoán khác nhau: nhiễu nhỏ hơn khoảng cách hẹp nhất 87 lần, không đủ sức lật dự đoán nào. Quy ra tiêu chí cho golden model C: sai số trên logit dưới `1e-03` là an toàn tuyệt đối, trên `1,9e-02` là có thể lật ảnh sát biên nhất. Chi tiết ở `roadmap-digit-recognition-v3.md` mục 6.

Lưu ý: `1,9e-02` là số của riêng `small_cnn.pth`. Train lại là phải chạy lại `study/dem_hoa_logit.py` để đo lại biên.